

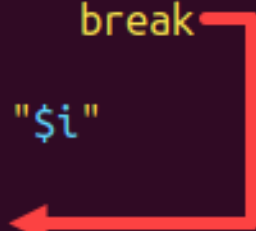


Higher Diploma in Computer Science

Computer Systems & Networks



```
1 #!/bin/bash
2
3 for i in {1..10}
4 do
5     if [[ $i == '2' ]]
6     then
7         echo "Number $i!"
8         break
9     fi
10    echo "$i"
11 done
12 echo "Done!"
```



break
continue

break statement

- The **break** and **continue** can be used to control the ANY loop type execution

(1) The break statement **break**

Output:

Number: 0

Number: 1

All Done!

```
i=0
while [ $i -lt 5 ]
do
    echo "Number: $i"
    ((i++))
    if [[ "$i" == '2' ]]; then
        break
    fi
done
echo 'All Done!'
```

break n – usage in nested loops

- The `break` statement on its own will break only **the loop that it was executed in**.
- To **break out of all the loops**, write the number of loops to break out of right after the break statement.

`$break n` #where n is the number of loops you wish to break from

continue statement

- The **break** and **continue** can be used to control the ANY loop type execution
- **continue** is a command that is used to allow you continue on to the next iteration of a loop

```
i=0
while [ $i -lt 5 ]
do
    ((i++))
    if [[ "$i" == '2' ]]; then
        echo "Skipping 2 and moving on.."
        continue
    fi
    echo "Number: $i"
done
echo 'All Done!'
```

```
karthick@OSTechNix:~$ /home/karthick/Documents/case.sh
```

```
Enter the first number(X) : 10
```

```
Enter the second number(Y) : 143
```

```
Addition => +
```

```
Subtract => -
```

```
Multiply => x
```

```
Division => /
```

```
Reminder => %
```

```
Choose any one opeartor : +
```

```
Addition of X and Y is 153
```

```
karthick@OSTechNix:~$ _
```



case...esac

case statement

Great alternative to nested if's

Useful when you wish to:

- create a menu
- Have a user choose a menu option
- Have a reaction based on their choice
- “lookup and execution-per-case statement”

case...esac statement

- ...is used to simplify complex conditions
- ... is a good alternative & more efficient than a multilevel if-then-else-fi statement.
- ...starts with **case** statement and closes by **esac** statement.

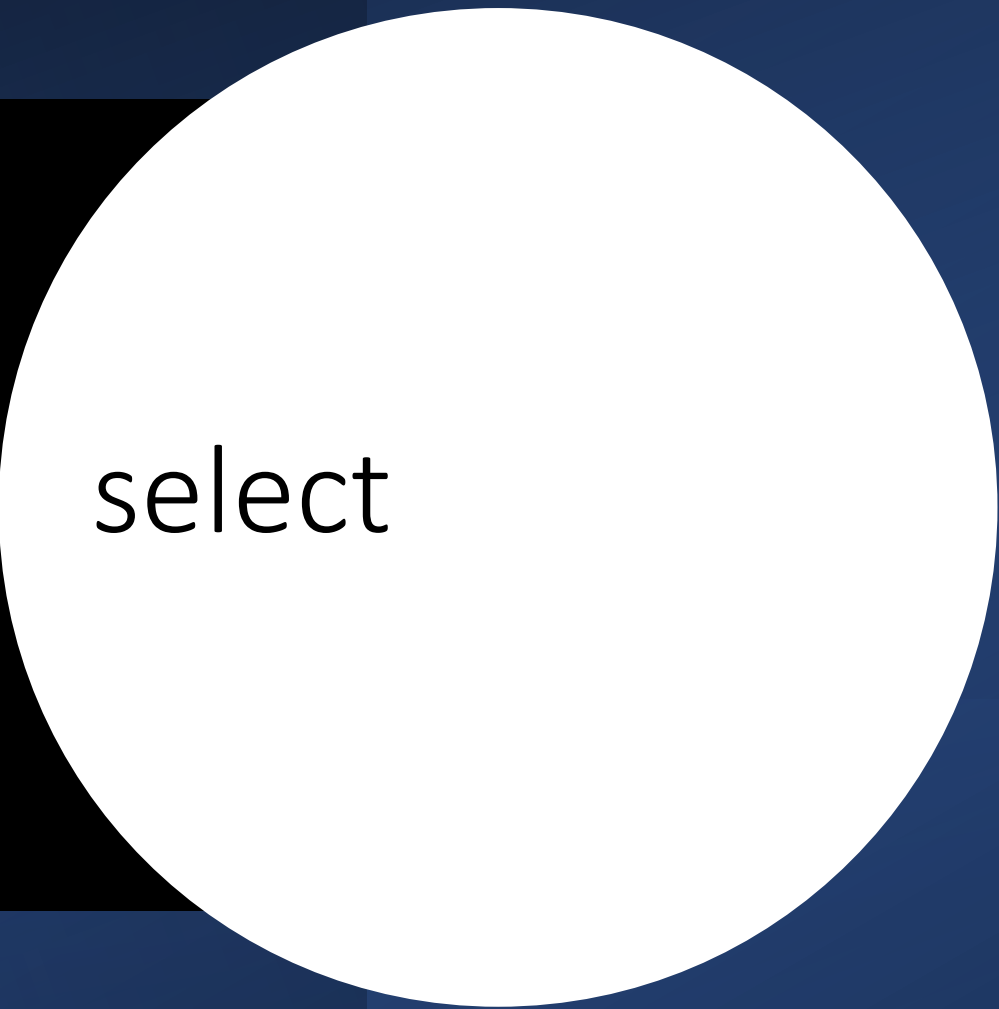
```
read word
case $word in
  add)
    Statement(s) to be executed if pattern1 matches
    ;;
  remove)
    Statement(s) to be executed if pattern2 matches
    ;;
  quit)
    Statement(s) to be executed if pattern3 matches
    ;;
  *)
    Default condition to be executed
    ;;
esac
```


case...esac statements

```
read userSelection
case $userSelection in
choice1)
    ACTIONS-TO-PERFORM
    ;;
choice2)
    ACTIONS-TO-PERFORM
    ;;
esac
```

```
#!/bin/bash
echo "Choose a contact to display information:"
echo "[C]hris Ramsey"
echo "[J]ames Gardner"
echo "[S]arah Snyder"
echo "[R]ose Armstrong"
read person
case "$person" in
    "C" | "c" ) echo "Chris Ramsey"
echo "cramsey@email.com"
echo "27 Railroad Dr. Bayside, NY";;
    "J" | "j" ) echo "James Gardner"
echo "jgardner@email.com"
echo "31 Green Street, Green Cove Springs, FL";;
    "S" | "s") echo "Sarah Snyder"
echo "ssnyder@email.com"
echo "8059 N. Hartford Court, Syosset, NY";;
    "R" | "r") echo "Rose Armstrong"
echo "rarmstrong@email.com"
echo "49 Woodside St., Oak Forest, IL";;
    *) echo "Contact doesn't exist.";;
esac
```

```
root@ubuntu:~# bash select-loop.sh
What do you do?
1) Student
2) Businessman
3) Professional
4) Fresher
Enter your choice ==> 1
You're still studying
Enter your choice ==> 2
Wow you work really hard for sure!
Enter your choice ==> 3
You've been a seasoned professional
Enter your choice ==> 4
You're new in the job market
Enter your choice ==> 5
Well, you need to enter something from the list!
Enter your choice ==> █
```



select

select statement to create a simple menu

- Gives user a list of options to choose
- Prompt's user for an input
- Syntax:

```
select var in <list>
do
    <commands>
done
```

```
options='Add Search Quit'
PS3='Choose your number: '
select name in $options
do
    if [ $name == 'Quit' ]
    then
        echo "you choose to quit, so long"
        break
    fi
echo You chose $name
done
```

A few points with use of *select*

- No error checking is done
- If the user hits **enter** without entering any data then the list of options will be displayed again.
- The loop will end when the **break** statement is issued.

Summary

while do done

Perform a set of commands while a test is true.

until do done

Perform a set of commands until a test is true.

for do done

Perform a set of commands for each item in a list.

break

Exit the currently running loop.

continue

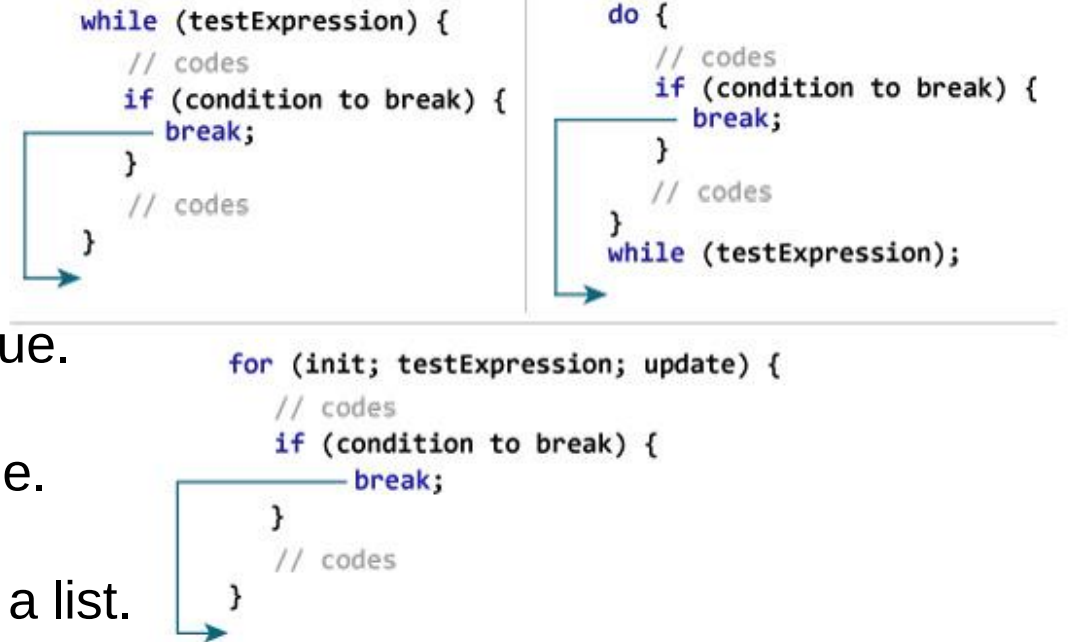
Stop this iteration of the loop and begin the next iteration.

select do done

Display a simple menu system for selecting items from a list.

case

The case statement executes any one block of commands, based on the outcome of a pattern match.



Return back to prompt Ctrl+C

